

Lab 3: Frontend, Ingress & Persistent Storage

From a Blank Folder to a Complete 3-Tier Stack Behind One Entry Point

What You Will Build

By the end of this lab you will have created an Angular application from scratch, containerized it, and deployed it to Kubernetes alongside your existing PostgreSQL and .NET Core API from Lab 2. A single Ingress will route browser traffic to either the frontend or the backend based on URL path. PostgreSQL will be backed by a PersistentVolume so its data survives Pod restarts — closing the gap deliberately left open in Lab 1.

Learning Objectives

- Scaffold and build an Angular application for production
- Containerize a static frontend using a multi-stage Docker build with nginx
- Install and understand the role of an Ingress Controller
- Configure path-based routing through a single Ingress resource
- Understand PersistentVolumes, PersistentVolumeClaims, and why Pods alone cannot retain data

Prerequisites

- Node.js and npm installed (node --version, npm --version)
- Angular CLI installed globally: npm install -g @angular/cli
- Lab 2 completed: postgres and api Pods running and reachable via /db-check
- Docker Desktop with Kubernetes enabled

Part A — Create the Angular Application

Step A1 — Scaffold a new Angular project

Run this from a clean working directory, outside your MyApi folder.

```
cd C:\k8s-training
ng new my-angular-app
```

When prompted, choose the defaults: no SSR/SSG (Server-Side Rendering) unless you specifically want it — this lab assumes a plain client-side rendered app, which is the simpler and more common starting point. Choose CSS for styling unless you have a preference.

This creates a my-angular-app folder with a working starter application.

```

Last login: Thu Jun 25 11:58:16 on ttys000
/dev/fd/12:25: command not found: compdef
vaibhavgupta@FVF998WQ85F-vaibhavgupta Day 44 June 25 % node --version
v24.16.0
vaibhavgupta@FVF998WQ85F-vaibhavgupta Day 44 June 25 % npm --version
11.13.0
vaibhavgupta@FVF998WQ85F-vaibhavgupta Day 44 June 25 % npm install -g @angular/cli
added 5 packages, removed 16 packages, and changed 272 packages in 44s

68 packages are looking for funding
  run 'npm fund' for details
npm notice
npm notice New minor version of npm available! 11.13.0 -> 11.17.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.17.0
npm notice To update run: npm install -g npm@11.17.0
npm notice
vaibhavgupta@FVF998WQ85F-vaibhavgupta Day 44 June 25 % cd k8s-training
vaibhavgupta@FVF998WQ85F-vaibhavgupta k8s-training % ng new my-angular-app
✔ Which stylesheet system would you like to use? Tailwind CSS [ https://tailwindcss.com ]
✔ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✔ Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE my-angular-app/.prettierrc (161 bytes)
CREATE my-angular-app/README.md (1465 bytes)
CREATE my-angular-app/.editorconfig (314 bytes)
CREATE my-angular-app/.gitignore (622 bytes)
CREATE my-angular-app/angular.json (1927 bytes)
CREATE my-angular-app/package.json (884 bytes)
CREATE my-angular-app/tsconfig.json (908 bytes)
CREATE my-angular-app/tsconfig.app.json (429 bytes)
CREATE my-angular-app/tsconfig.spec.json (441 bytes)
CREATE my-angular-app/postcssrc.json (53 bytes)
CREATE my-angular-app/.vscode/extensions.json (138 bytes)
CREATE my-angular-app/.vscode/launch.json (478 bytes)
CREATE my-angular-app/.vscode/tasks.json (978 bytes)
CREATE my-angular-app/src/main.ts (222 bytes)
CREATE my-angular-app/src/index.html (298 bytes)
CREATE my-angular-app/src/styles.css (184 bytes)
CREATE my-angular-app/src/app/app.css (8 bytes)
CREATE my-angular-app/src/app/app.spec.ts (681 bytes)
CREATE my-angular-app/src/app/app.ts (296 bytes)
CREATE my-angular-app/src/app/app.html (20144 bytes)
CREATE my-angular-app/src/app/app.config.ts (313 bytes)
CREATE my-angular-app/src/app/app.routes.ts (77 bytes)
CREATE my-angular-app/public/favicon.ico (15086 bytes)
✔ Packages installed successfully.
Directory is already under version control. Skipping initialization of git.
vaibhavgupta@FVF998WQ85F-vaibhavgupta k8s-training %

```

Step A2 — Add a simple call to your backend API

Open `my-angular-app/src/app/app.ts` (or `app.component.ts`, depending on your Angular CLI version) and replace its contents with the following. This adds a button that calls your API's `/db-check` endpoint and displays the result, proving the full chain works once deployed.

`src/app/app.ts`

```

import { Component, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-root',
  standalone: true,
  template: `
    <div style="font-family: sans-serif; padding: 2rem;">
      <h1>Kubernetes 3-Tier Demo</h1>
      <button (click)="checkDb()">Check Database Connection</button>
      <pre>{{ result() }}</pre>
    </div>
  `,
})
export class App {
  result = signal('Click the button to test the API -> Postgres connection');

  constructor(private http: HttpClient) {}

```

```
checkDb() {  
  this.http.get('/api/db-check').subscribe({  
    next: (res) => this.result.set(JSON.stringify(res, null, 2)),  
    error: (err) => this.result.set('Error: ' + JSON.stringify(err.message)),  
  });  
}
```

Step A3 — Enable HttpClient

Open `src/app/app.config.ts` and ensure `provideHttpClient()` is included, so the `HttpClient` used above actually works.

`src/app/app.config.ts`

```
import { ApplicationConfig } from '@angular/core';  
import { provideHttpClient } from '@angular/common/http';  
  
export const appConfig: ApplicationConfig = {  
  providers: [provideHttpClient()],  
};
```

Why the URL is `/api/db-check`, not the full backend address

The Angular app deliberately calls a relative path, not a hostname or port. Once deployed, the Ingress we configure in Part C will intercept any request starting with `/api` and forward it to the backend Service. The frontend code never needs to know the backend's actual location.

Step A4 — Build for production

```
cd my-angular-app  
ng build
```

Once complete, check the exact output path — this varies by Angular version:

```
dir dist\my-angular-app
```

Important

Angular 17 and later nest build output under a `/browser` subfolder (`dist/my-angular-app/browser`) to support server-side rendering setups. Older versions place files directly under `dist/my-angular-app`. Confirm which structure you have now -- you will need the exact path in Step B1.

```
vaibhavgupta@FVF0905WQ85F-vaibhavgupta k8s-training % cd my-angular-app
ng build

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

[ ] No
Global setting: enabled
Local setting: disabled
Effective status: disabled
Initial chunk files | Names | Raw size | Estimated transfer size
main-KCHSQCIG.js | main | 124.38 kB | 36.54 kB
styles-Z22TQY4R.css | styles | 4.52 kB | 1.40 kB
| Initial total | 128.90 kB | 37.94 kB

Application bundle generation complete. [34.491 seconds] - 2026-06-25T09:48:46.007Z

Output location: /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 44 June 25/k8s-training/my-angular-app/dist/my-angular-app
vaibhavgupta@FVF0905WQ85F-vaibhavgupta my-angular-app % █
```

Part B — Containerize the Angular App

Step B1 — Create the Dockerfile

This is a multi-stage build, the same pattern used for the .NET API: stage one compiles the app, stage two serves only the compiled static files using nginx. Adjust the COPY path in stage two to match what you found in Step A4.

Dockerfile

```
# Stage 1: Build the Angular app
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve with nginx
FROM nginx:alpine
# Adjust this path to match your actual dist output from Step A4
COPY --from=build /app/dist/my-angular-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Step B2 — Create nginx.conf

This single setting is the most commonly missed step when containerizing any single-page application. Without it, refreshing the browser on any route other than the homepage returns a 404, because nginx looks for a literal file at that path and finds none.

nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Place both Dockerfile and nginx.conf in the root of my-angular-app, alongside package.json.

Step B3 — Build the image

```
docker build -t myangularapp:latest .
```

```

vaibhavgupta@FVF0985W085F-vaibhavgupta my-angular-app % docker build -t myangularapp:latest .
[+] Building 218.3s (15/15) FINISHED                                docker:desktop-linu
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 414B
=> [internal] load metadata for docker.io/library/node:22
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/6] FROM docker.io/library/node:22@sha256:c681a46abb4d2ab80a9dc3da208d50d1122642d53f17a101926ace71e5a9bf1c
=> => resolve docker.io/library/node:22@sha256:c681a46abb4d2ab80a9dc3da208d50d1122642d53f17a101926ace71e5a9bf1c
=> => sha256:0bf6c1c77c3e989e08991a446527d314540b0e9ac767c3dcd48a4d8a76eaf 1.29MB / 1.29MB
=> => sha256:4762c4203ab3e4556711d44cfc62e89a9fb4bca24c1bdd2b95b624d8d8bdeb 447B / 447B
=> => sha256:38e09163b572c69a3fa474684abdb810180cb27cd4ab595c108706efaceb9d5e 58.68MB / 58.68MB
=> => sha256:8cae40c24fa4bb4f22bb94674a5eb530bfff1cfd2e063ec20853ecb9a5589556 203.13MB / 203.13MB
=> => sha256:8f52dc8996e4f73a7e828bc9a2076243f49b3aa8fc5d4b97b3973bb2441893ee 3.32KB / 3.32KB
=> => sha256:5330b0e91970911b4c87a1d3be9ad0e1a3308fcb7240c122bbae0220af6cf 64.49MB / 64.49MB
=> => sha256:03ebca214f1a4b66acfd8bd20aa3ee139d1747885ef4b0f3d07aa2a68459230 23.61MB / 23.61MB
=> => sha256:0fb1189398e2e4b474d43aac6502510d0da8318e70137a377c21087f198814db 48.39MB / 48.39MB
=> => extracting sha256:0fb1189398e2e4b474d43aac6502510d0da8318e70137a377c21087f198814db
=> => extracting sha256:03ebca214f1a4b66acfd8bd20aa3ee139d1747885ef4b0f3d07aa2a68459230
=> => extracting sha256:5330b0e91970911b4c87a1d3be9ad0e1a3308fcb7240c122bbae0220af6cf
=> => extracting sha256:8cae40c24fa4bb4f22bb94674a5eb530bfff1cfd2e063ec20853ecb9a5589556
=> => extracting sha256:8f52dc8996e4f73a7e828bc9a2076243f49b3aa8fc5d4b97b3973bb2441893ee
=> => extracting sha256:38e09163b572c69a3fa474684abdb810180cb27cd4ab595c108706efaceb9d5e
=> => extracting sha256:8cf6c1c77c3e989e08991a446527d314540b0e9ac767c3d3ce8a4d85a76e4f
=> => extracting sha256:4762c4203ab3e4556711d44cfc62e89a9fb4bca24c1bdd2b95b624d8d8bdeb
=> CAHED [stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:54f2a904c251d5a34adf545a72d32515a15e08418dae026ae23be2e18c66fefa
=> => resolve docker.io/library/nginx:alpine@sha256:54f2a904c251d5a34adf545a72d32515a15e08418dae026ae23be2e18c66fefa
=> [internal] load build context
=> => transferring context: 1.81MB
=> [build 2/6] WORKDIR /app
=> [build 3/6] COPY package*.json ./
=> [build 4/6] RUN npm install
=> [build 5/6] COPY . .
=> [build 6/6] RUN npm run build
=> [stage-1 2/3] COPY --from=build /app/dist/my-angular-app/browser /usr/share/nginx/html
=> [stage-1 3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:85040a4c36ce3485663ed49454078883a1e587763560239c236a04b5ea5671d7
=> => exporting config sha256:88ce765d970b4933e9c81cc269a87e041d6a9f1acda7f087cb2fbc9367b610
=> => exporting attestation manifest sha256:f07d43fca725f241667fcd807e2d36fb91fbf41a1b81a4ad8c9279081acee00
=> => exporting manifest list sha256:04b643e5f8e372516b836cbca19194222169bb16864ae013f9d2278f121f2882d
=> => naming to docker.io/library/myangularapp:latest
=> => unpacking to docker.io/library/myangularapp:latest

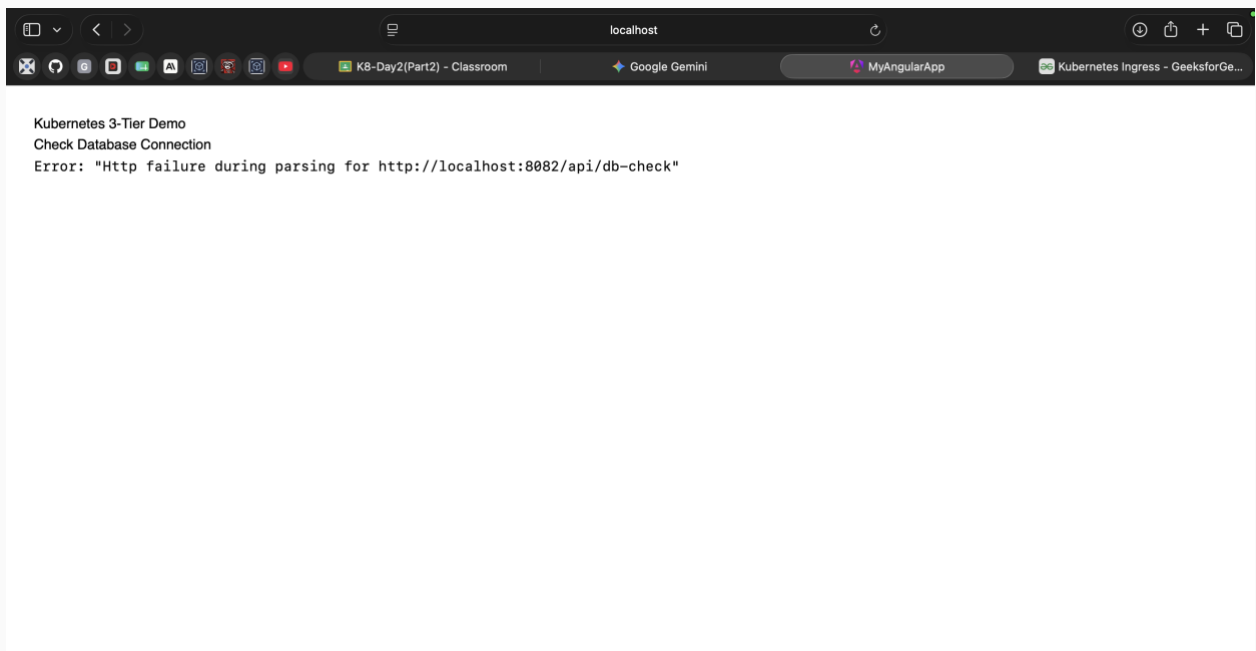
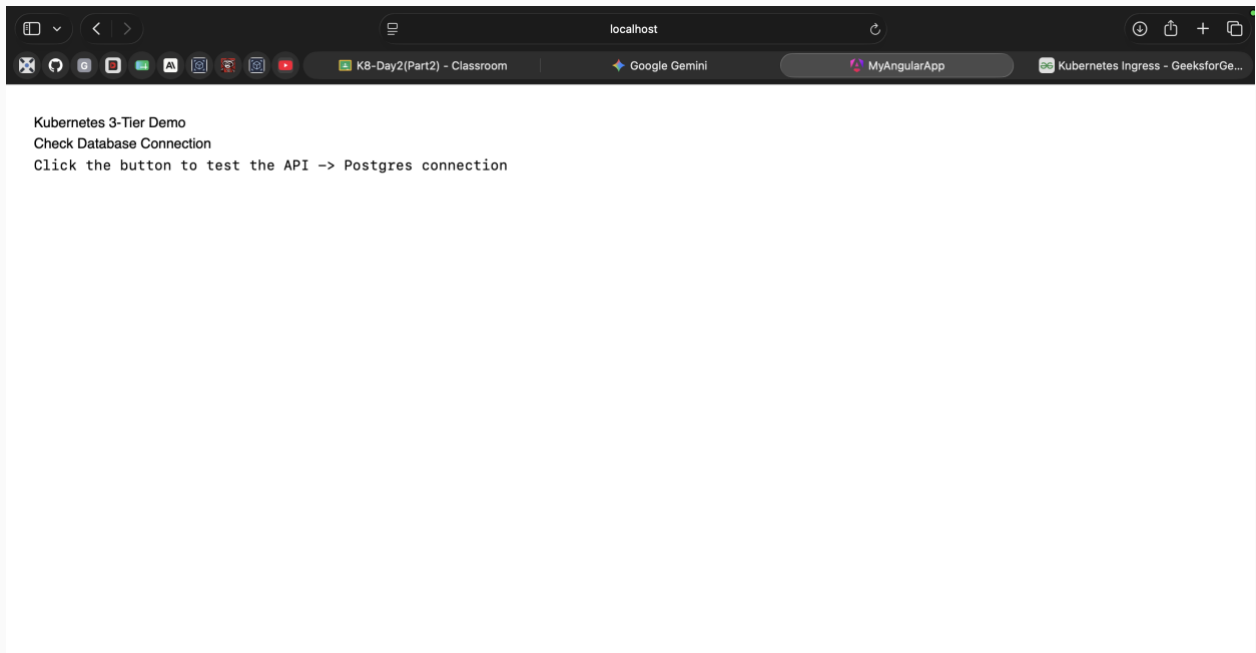
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qu947r08qxdxumhtfc2n1laca
vaibhavgupta@FVF0985W085F-vaibhavgupta my-angular-app %

```

Step B4 — Smoke-test locally

```
docker run -p 8082:80 myangularapp:latest
```

Open <http://localhost:8082/> in a browser. You should see the page with the "Check Database Connection" button. Clicking it will fail right now with a network error -- that is expected, since `/api/db-check` has nothing to route to outside the cluster. Stop the container with `Ctrl+C` once confirmed the page loads.



Step B5 — Make the image visible to your cluster (kind only)

Skip this step if you chose Kubeadm

This step only applies if Docker Desktop's Kubernetes was set up using the kind engine. If you chose Kubeadm instead, your cluster shares the host's Docker image cache directly, and this step is unnecessary -- continue straight to Part C.

A kind cluster runs its node as a separate container with its own isolated container runtime and image storage, entirely separate from your host machine's Docker image cache. Building an image with docker build on your host does not automatically make it visible inside the kind node. Skipping this step is the most common cause of an ImagePullBackOff or ErrImagePull error in Part C.

First, find the exact name of your node container:

```
docker ps
```

Look for a container whose name ends in -control-plane (for example, desktop-control-plane). Then load the image directly into that node's containerd runtime:

```
docker save myangularapp:latest | docker exec -i desktop-control-plane ctr -n=k8s.io images import -
```

Replace desktop-control-plane with your actual node container name if different. Repeat this step any time you rebuild the image -- the kind node never picks up a rebuild automatically.

```
2026/06/25 11:02:19 [notice] 1#1: signal 17 (SIGCHLD) received from 32
2026/06/25 11:02:19 [notice] 1#1: worker process 32 exited with code 0
2026/06/25 11:02:19 [notice] 1#1: exit
vaibhav@vaibhav:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
vaibhav@vaibhav:~$ docker save myangularapp:latest | docker exec -i desktop-control-plane ctr -n=k8s.io images import -
docker.io/library/myangularapp:latest saved
application/vnd.oci.image.index.v1+json sha256:ced77b2d6b6117736b45fd7fa6bc0bc71a4fe569589f4aea9607df36095ca4d
Importing      elapsed: 5.1 s total:  0.0 B (0.0 B/s)
vaibhav@vaibhav:~$ docker ps
```

Part C — Deploy the Frontend to Kubernetes

Step C1 — Create the frontend manifest

03-frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
```

```
metadata:
  labels:
    app: frontend
spec:
  containers:
    - name: frontend
      image: myangularapp:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: "50m"
          memory: "64Mi"
        limits:
          cpu: "200m"
          memory: "256Mi"
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 10
        periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Note the replica count of 2, not 1. Serving static files is cheap and entirely stateless, so there is no reason to run only a single copy from the start.

Step C2 — Deploy it

```
kubectl apply -f 03-frontend.yaml
kubectl get pods --watch
```

Wait until both frontend pods show 1/1 Running, then press Ctrl+C.

```
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
vaibhavgupta@FVF998WQ85F~vaibhavgupta my-angular-app % kubectl apply -f 03-frontend.yaml
deployment.apps/frontend created
service/frontend created
vaibhavgupta@FVF998WQ85F~vaibhavgupta my-angular-app % kubectl get pods --watch
NAME                                READY   STATUS    RESTARTS   AGE
api-65c4f494df-vkrch                1/1     Running   0           3h43m
frontend-6695bbf6b8-7pqzz           0/1     Running   0           11s
frontend-6695bbf6b8-9cmx7           0/1     Running   0           11s
my-app-59854d5646-zc8jc             1/1     Running   0           4h11m
postgres-9f64456b7d-p7dvj          1/1     Running   0           3h56m
frontend-6695bbf6b8-7pqzz           1/1     Running   0           13s
frontend-6695bbf6b8-9cmx7           1/1     Running   0           14s
```

Part D — Install an Ingress Controller

Defining an Ingress resource by itself does nothing on its own -- it is only a routing rule. Something must actually read that rule and implement the routing: that something is the Ingress Controller, a real running piece of software. Docker Desktop does not install one by default, so this step installs it explicitly.

Step D1 — Install the nginx Ingress Controller

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.11.2/deploy/static/provider/cloud/deploy.yaml
```

This deploys the controller into its own namespace, separate from your application Pods.

Step D2 — Wait for the controller to be ready

```
kubectl get pods -n ingress-nginx --watch
```

Wait for a pod named `ingress-nginx-controller-...` to reach `1/1 Running`. This can take 30-60 seconds. Press `Ctrl+C` once ready.

```
configmap/ingress-nginx-controller created
service/ingress-nginx-controller created
service/ingress-nginx-controller-admission created
deployment.apps/ingress-nginx-controller created
job.batch/ingress-nginx-admission-create created
job.batch/ingress-nginx-admission-patch created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/ingress-nginx-admission created
vaibhavgupta@FVF698WQ85F~vaibhavgupta my-angular-app % kubectl get pods -n ingress-nginx --watch
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-zb8bb 0/1     Completed 0          98s
ingress-nginx-admission-patch-2w4qg 0/1     Completed 1          98s
ingress-nginx-controller-6d6657db77-478g9 1/1     Running   0          98s
```

Part E — Configure the Ingress

A subtle bug worth understanding up front

A single Ingress resource with one `rewrite-target` annotation covering both the frontend and backend paths looks reasonable, but breaks the frontend in a specific way: annotations apply to the entire Ingress resource, not to individual paths within it. The `rewrite-target` rule, intended only to strip `/api`, ends up rewriting every request that flows through that Ingress -- including requests for the Angular app's own JavaScript and CSS files. The browser then receives the wrong content for those files and throws a MIME-type error. The fix is to use two separate Ingress resources, so the rewrite only ever applies to API traffic.

Step E1 — Create the frontend Ingress

This resource has no rewrite annotation at all. Requests pass through to the frontend Service exactly as received.

03-frontend-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Step E2 — Create the API Ingress

This resource keeps the rewrite-target annotation, but it is now scoped to its own Ingress resource and therefore only ever affects API traffic. It strips the /api prefix before forwarding, so the request arrives at the backend exactly as the route definitions expect (e.g. /db-check, not /api/db-check).

03-api-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: api
                port:
                  number: 8080
```

Step E3 — Apply and verify both

```
kubectl apply -f 03-frontend-ingress.yaml
kubectl apply -f 03-api-ingress.yaml
kubectl get ingress
```

Expect to see two separate Ingress entries listed, frontend-ingress and api-ingress, both showing localhost under ADDRESS.

```
vaibhav@FVF995WQ85F-vaibhav:~$ kubectl apply -f 03-frontend-ingress.yaml
ingress.networking.k8s.io/frontend-ingress created
vaibhav@FVF995WQ85F-vaibhav:~$ kubectl apply -f 03-api-ingress.yaml
ingress.networking.k8s.io/api-ingress created
vaibhav@FVF995WQ85F-vaibhav:~$ kubectl get ingress
NAME                CLASS    HOSTS      ADDRESS      PORTS      AGE
api-ingress         nginx    *          172.21.0.5   80         37s
frontend-ingress    nginx    *          172.21.0.5   80         46s
vaibhav@FVF995WQ85F-vaibhav:~$ kubectl get ingress
NAME                CLASS    HOSTS      ADDRESS      PORTS      AGE
api-ingress         nginx    *          172.21.0.5   80         40s
frontend-ingress    nginx    *          172.21.0.5   80         48s
vaibhav@FVF995WQ85F-vaibhav:~$
```

Step E4 — Test the full stack through one entry point

Use an Incognito/InPrivate window for this test, to rule out any stale cached files from earlier attempts.

1. Open <http://localhost/> in a fresh Incognito/InPrivate window -- this should load your Angular app with no console errors.
2. Click the "Check Database Connection" button.
3. Confirm it returns a successful JSON response with the Postgres version, instead of the network error seen in Step B4.



Kubernetes 3-Tier Demo
Check Database Connection
Click the button to test the API -> Postgres connection

Kubernetes 3-Tier Demo
Check Database Connection

```
{
  "status": "connected",
  "database": "appdb",
  "host": "postgres",
  "postgresVersion": "PostgreSQL 16.14 (Debian 16.14-1.pgdg13+1) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-b"
}
```

This proves the entire chain end-to-end: browser, to Ingress, to frontend or backend Service by path, to the API Pod, to the Postgres Service by DNS name, to the Postgres Pod.

Part F — Persistent Storage for PostgreSQL

In Lab 1, deleting a Pod proved Kubernetes' self-healing behavior -- but it also proved something uncomfortable: the replacement Pod got a completely empty filesystem. Any data PostgreSQL had written was gone. This part closes that gap.

Step F1 — Understand the two objects involved

PersistentVolume (PV) vs PersistentVolumeClaim (PVC)

A PersistentVolume is a piece of real storage in the cluster -- a chunk of disk space, independent of any particular Pod's lifecycle.

A PersistentVolumeClaim is a request for storage made by a Pod -- "I need 1Gi of space." Kubernetes matches claims to available volumes.

This separation means storage survives even if the Pod requesting it is deleted and recreated. The new Pod's claim binds to the same existing volume, not a new, empty one.

Step F2 — Create the PersistentVolumeClaim

On Docker Desktop, a default StorageClass automatically provisions the underlying volume for you, so only a PVC needs to be defined explicitly.

03-postgres-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

ReadWriteOnce means the volume can be mounted as read-write by a single node at a time -- the correct mode for a single-instance database like this one.

Step F3 — Update the PostgreSQL Deployment to use it

Two additions to the existing Deployment from Lab 2: a volumeMounts entry inside the container, and a volumes entry at the Pod spec level referencing the PVC.

03-postgres-with-storage.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
name: postgres
labels:
  app: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: appdb
            - name: POSTGRES_USER
              value: appuser
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: DB_PASSWORD
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

Why the PGDATA environment variable is added

Postgres' own data directory and the volume mount point both default to `/var/lib/postgresql/data`. Mounting a fresh volume directly there can conflict with a hidden lost+found directory some storage drivers create. Pointing PGDATA at a subfolder (pgdata) inside the mount avoids that conflict entirely.

Step F4 — Apply the updated storage configuration

```
kubectl apply -f 03-postgres-pvc.yaml
kubectl apply -f 03-postgres-with-storage.yaml
kubectl get pvc
kubectl get pods --watch
```

Confirm the PVC shows STATUS Bound, and the postgres pod restarts and reaches 1/1 Running again.

```
api-ingress      nginx      *      172.21.0.5      80      99s
frontend-ingress nginx      *      172.21.0.5      80      107s
vaibhavgupta@FVF0909WQ05F-vaibhavgupta my-angular-app % kubectl apply -f 03-postgres-pvc.yaml
persistentvolumeclaim/postgres-pvc created
vaibhavgupta@FVF0909WQ05F-vaibhavgupta my-angular-app % kubectl apply -f 03-postgres-with-storage.yaml
deployment.apps/postgres configured
service/postgres unchanged
vaibhavgupta@FVF0909WQ05F-vaibhavgupta my-angular-app % kubectl get pvc
NAME      STATUS  VOLUME      CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
postgres-pvc  Bound  pvc-2c80c19c-8bde-4a9d-8c19-304c80981526  1Gi         RWO           standard              <unset>              29s
vaibhavgupta@FVF0909WQ05F-vaibhavgupta my-angular-app % kubectl get pods --watch
NAME      READY  STATUS  RESTARTS  AGE
api-65c4f494df-vkrch  1/1    Running  0          3h57m
frontend-6695bbf6b8-7pzzz  1/1    Running  0          14m
frontend-6695bbf6b8-7cmz7  1/1    Running  0          14m
my-app-59854d5446-zc8jc  1/1    Running  0          4h25m
postgres-848f49ccd-hnld8  1/1    Running  0          42s
```

Step F5 — Prove data survives a Pod deletion

4. Open a shell into the postgres pod and connect with psql.
5. Create a table and insert one row.
6. Exit, then delete the postgres pod directly.
7. Wait for the replacement pod to reach Running.
8. Connect again and confirm the table and row still exist.

```
kubectl exec -it postgres-848f49ccd-hnld8 -- psql -U appuser -d appdb
```

```
-- inside psql:
CREATE TABLE proof (id SERIAL PRIMARY KEY, note TEXT);
INSERT INTO proof (note) VALUES ('data survived a pod restart');
\q
```

```
kubectl delete pod postgres-848f49ccd-hnld8
kubectl get pods --watch
```

Once the new pod is Running, reconnect and check:

```
kubectl exec -it postgres-848f49ccd-smbh8 -- psql -U appuser -d appdb -c "SELECT
* FROM proof;"
```

The row should still be there. Compare this directly to Lab 1, where deleting the unmounted Postgres pod lost everything -- the only difference now is the PersistentVolumeClaim.

```

^C
vaibhavgupta@FVF0905WQ05F-vaibhavgupta my-angular-app % kubectl exec -it postgres-848f49ccd-hnld8 -- psql -U appuser -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# -- inside psql:
CREATE TABLE proof (id SERIAL PRIMARY KEY, note TEXT);
INSERT INTO proof (note) VALUES ('data survived a pod restart');
\q
CREATE TABLE
INSERT 0 1
vaibhavgupta@FVF0905WQ05F-vaibhavgupta my-angular-app % kubectl delete pod postgres-848f49ccd-hnld8
pod "postgres-848f49ccd-hnld8" deleted from default namespace
vaibhavgupta@FVF0905WQ05F-vaibhavgupta my-angular-app % kubectl get pods --watch
NAME                                READY   STATUS    RESTARTS   AGE
api-d5c4f494df-wk7ch               1/1     Running   0           4h
frontend-6695bbf6b8-7pazz          1/1     Running   0           17m
frontend-6695bbf6b8-9cmx7          1/1     Running   0           17m
my-app-59854d5646-zc8jc             1/1     Running   0           4h28m
postgres-848f49ccd-smbh8            1/1     Running   0           11s
frontend-6695bbf6b8-9cmx7          1/1     Running   0           17m
postgres-848f49ccd-smbh8            1/1     Running   0           62s
^C
vaibhavgupta@FVF0905WQ05F-vaibhavgupta my-angular-app % kubectl exec -it postgres-848f49ccd-smbh8 -- psql -U appuser -d appdb -c "SELECT * FROM proof;"
id |
----+-----
 1 | data survived a pod restart
(1 row)

vaibhavgupta@FVF0905WQ05F-vaibhavgupta my-angular-app %

```

Troubleshooting Reference

Symptom	Likely Cause / Fix
nginx serves a blank page or 404 for any route	The dist output path in the Dockerfile does not match your actual Angular build output. Re-check with <code>dir dist/my-angular-app</code> and adjust the COPY line.
Refreshing on a deep link returns 404	nginx.conf is missing or its <code>try_files</code> directive was not applied. Confirm the file was actually copied into the image.
frontend pod shows ImagePullBackOff or ErrImagePull	If using kind, the image was built on the host but never loaded into the kind node's separate image store. Repeat Step B5.
kubectl get ingress shows no ADDRESS	The Ingress Controller is not yet ready, or did not install correctly. Re-check <code>kubectl get pods -n ingress-nginx</code> .
Browser console shows a MIME-type error for a .js file, page renders blank	A <code>rewrite-target</code> annotation is being applied to the frontend's own Ingress rule, not just the API rule. Confirm the frontend Ingress (frontend-ingress) has no rewrite annotation at all -- only api-ingress should have one.
/api/db-check returns 404 through the Ingress but works via port-forward	The <code>rewrite-target</code> annotation or path regex on api-ingress does not match. Confirm the path is exactly <code>/api(/ \$)(.*)</code> and the annotation is <code>/ \$2</code> .
PVC stuck in Pending	No StorageClass is available to satisfy the claim. On Docker Desktop this is rare, but confirm with <code>kubectl get storageclass</code> that a default class exists.
Postgres pod fails to start after adding the volume	Check <code>kubectl logs <pod-name></code> for a permissions or lost+found error; confirm PGDATA is set to a subfolder, not the mount root.

Key takeaway

Three separate problems, three separate Kubernetes primitives: Ingress solved "how do two different apps share one entry point," the Ingress Controller solved "who actually implements that routing," and the PersistentVolumeClaim solved "how does data outlive a Pod." None of these existed in Lab 1 by accident -- each was introduced only once its absence became a felt, concrete problem.